

ftrace and trace-cmd



ftrace

- *ftrace* is a tracing framework within the kernel which stands for “Function Tracer”.
- It offers a wide range of tracing capabilities allowing to observe the system behavior.
  - Trace static tracepoints already inserted at various locations in the kernel (scheduler, interrupts, etc).
  - Relies on GCC mcount() capability and kernel code patching mechanism to call *ftrace* tracing handlers.
- All traces are recorded in a ring buffer that is optimized for tracing.
- Uses *tracefs* filesystem to control and display tracing events.
  - `# mount -t tracefs nodev /sys/kernel/tracing`.
- *ftrace* support must be enabled in the kernel using CONFIG_FTRACE=y.
- CONFIG_DYNAMIC_FTRACE allows to have a zero overhead tracing support.



ftrace files

- *ftrace* controls are exposed through some specific files located under `/sys/kernel/tracing`.
  - `current_tracer`: Current tracer that is used.
  - `available_tracers`: List of available tracers that are compiled in the kernel.
  - `tracing_on`: Enable/disable tracing.
  - `trace`: Acquired trace in human readable format. Format will differ depending on the tracer used.
  - `trace_pipe`: same as `trace`, but each read consumes the trace as it is read.
  - `trace_marker_raw`: Emit comments from userspace in the trace buffer.
  - `set_ftrace_filter`: Filter some specific functions.
  - `set_graph_function`: Graph only the specified functions child.
- Many other files are exposed, see trace/ftrace.
- *trace-cmd* CLI and *Kernelshark* GUI tools allow to record and visualize tracing data more easily.



ftrace tracers

- ftrace provides several “tracers” which allow to trace different things.
- The tracer to be used should be written to the `current_tracer` file
  - `nop`: Trace nothing, used to disable all tracing.
  - `function`: Trace all kernel functions that are called.
  - `function_graph`: Similar to `function` but traces both entry and exit.
  - `hwlat`: Trace hardware latency.
  - `irqsoff`: Trace sections where interrupts are disabled.
  - `branch`: Trace likely()/unlikely() prediction errors.
  - `mmiotrace`: Trace all accesses to the hardware (`read[bwlq]/write[bwlq]`).
- **Warning: Some tracers can be expensive!**

```
# echo "function" > /sys/kernel/tracing/current_tracer
```



function_graph tracer report example

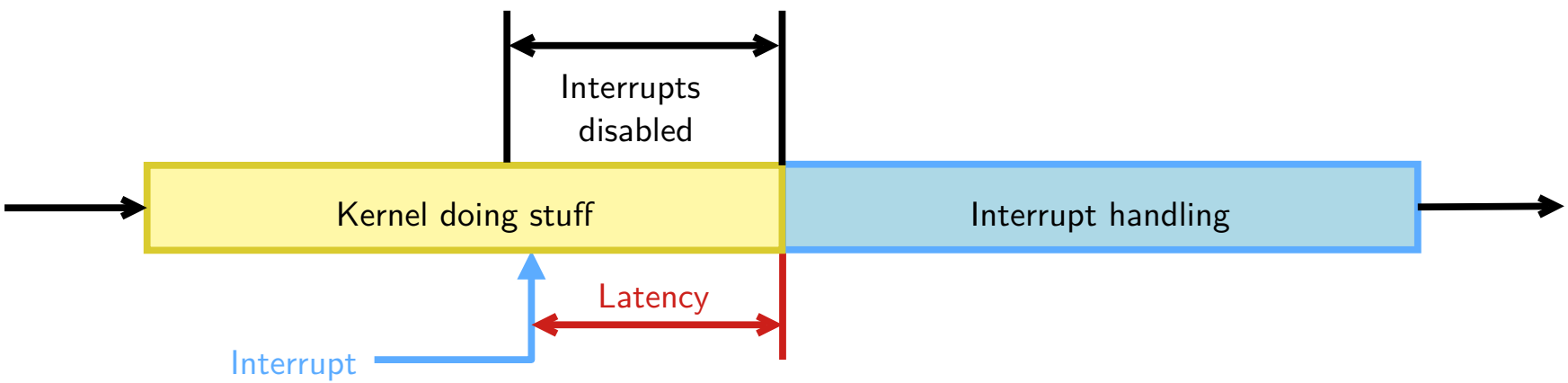
- The *function_graph* traces all the function that executed and their associated callgraphs
- Will display the process, CPU, timestamp and function graph:

```
$ trace-cmd report
...
dd-113   [000]   304.526590: funcgraph_entry:                   |  sys_write() {
dd-113   [000]   304.526597: funcgraph_entry:                   |    ksys_write() {
dd-113   [000]   304.526603: funcgraph_entry:                   |      __fdget_pos() {
dd-113   [000]   304.526609: funcgraph_entry:       6.541 us    |        __fget_light();
dd-113   [000]   304.526621: funcgraph_exit:      + 18.500 us   |      }
dd-113   [000]   304.526627: funcgraph_entry:                   |      vfs_write() {
dd-113   [000]   304.526634: funcgraph_entry:       6.334 us    |        rw_verify_area();
dd-113   [000]   304.526646: funcgraph_entry:       6.208 us    |        write_null();
dd-113   [000]   304.526658: funcgraph_entry:       6.292 us    |        __fsnotify_parent();
dd-113   [000]   304.526669: funcgraph_exit:      + 43.042 us   |      }
dd-113   [000]   304.526675: funcgraph_exit:      + 78.833 us   |    }
dd-113   [000]   304.526680: funcgraph_exit:      + 91.291 us   |  }
dd-113   [000]   304.526689: funcgraph_entry:                   |  sys_read() {
dd-113   [000]   304.526695: funcgraph_entry:                   |    ksys_read() {
dd-113   [000]   304.526702: funcgraph_entry:                   |      __fdget_pos() {
dd-113   [000]   304.526708: funcgraph_entry:       6.167 us    |        __fget_light();
dd-113   [000]   304.526719: funcgraph_exit:      + 18.083 us   |      }
```



irqsoff tracer

- ftrace *irqsoff* tracer allows to trace the irqs latency due to interrupts being disabled for too long.
- Helpful to find why interrupts have high latencies on a system.
- This tracer will record the longest trace with interrupts being disabled.
- This tracer needs to be enabled with CONFIG_IRQSOFF_TRACER=y.
  - `preemptoff`, `premptirqsoff` tracers also exist to trace section of code were preemption is disabled.





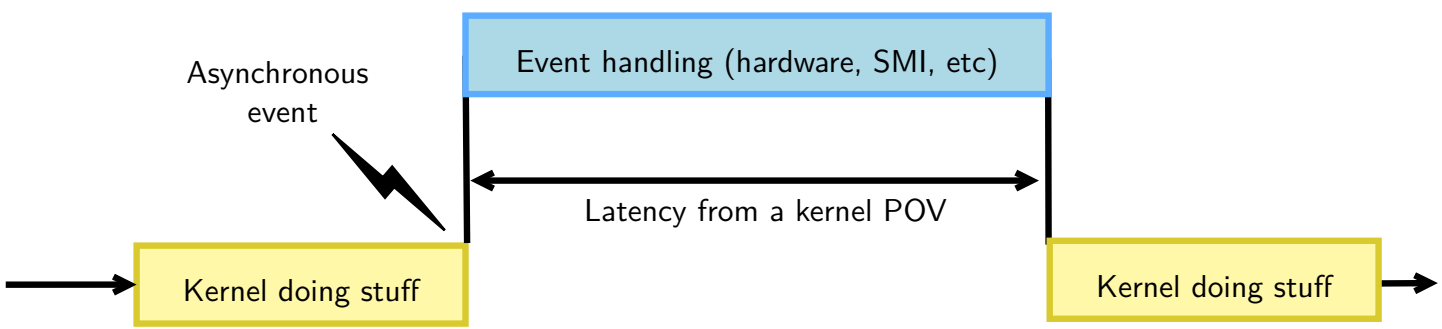
irqsoff: report example

```
# latency: 276 us, #104/104, CPU#0 | (M:preempt VP:0, KP:0, SP:0 HP:0 #P:2)
#    -----------------
#    | task: stress-ng-114 (uid:0 nice:0 policy:0 rt_prio:0)
#    -----------------
#  => started at: __irq_usr
#  => ended at:   irq_exit
#
#
#                    _------=> CPU#
#                   / _-----=> irqs-off
#                  | / _----=> need-resched
#                  || / _---=> hardirq/softirq
#                  ||| / _--=> preempt-depth
#                  |||| /     delay
#  cmd     pid     ||||| time  |   caller
#       /         |||||      |   /
stress-n-114       0d...    2us : __irq_usr
stress-n-114       0d...    7us : gic_handle_irq <-__irq_usr
stress-n-114       0d...   10us : __handle_domain_irq <-gic_handle_irq
...
stress-n-114       0d...  270us : __local_bh_disable_ip <-__do_softirq
stress-n-114       0d.s.  275us : __do_softirq <-irq_exit
stress-n-114       0d.s.  279us+: tracer_hardirqs_on <-irq_exit
stress-n-114       0d.s.  290us : <stack trace>
```



Hardware latency detector

- ftrace *hwlat* tracer will help to find if the hardware generates latency.
  - System Management interrupts for instance are non maskable and directly trigger some firmware support feature, suspending CPU execution.
  - Interrupts handled by secure monitor can also cause this kind of latency.
- If some latency is found with this tracer, the system is probably not suitable for real time usage.
- Uses a single core looping while interrupts are disabled and measuring the time elapsed between two consecutive time reads.
- Needs to be builtin the kernel with CONFIG_HWLAT_TRACER=y.





trace-cmd

- *trace-cmd* is a tool written by Steven Rostedt which allows interacting with *ftrace* (man 1 trace-cmd).
- The tracers supported by *trace-cmd* are those exposed by ftrace.
- *trace-cmd* offers multiple commands:
  - *list*: List available plugins/events that can be recorded.
  - *record*: Record a trace into the file `trace.dat`.
  - *report*: Display `trace.dat` acquisition results.



trace-cmd examples (1/3)

- List available tracers

```
$ trace-cmd list -t
blk mmiotrace function_graph function nop
```

- List available events

```
$ trace-cmd list -e
...
migrate:mm_migrate_pages_start
migrate:mm_migrate_pages
tlb:tlb_flush
syscalls:sys_exit_process_vm_writev
...
```

- List available functions for filtering with `function` and `function_graph` tracers

```
$ trace-cmd list -f
...
wait_for_initramfs
__ftrace_invalid_address___64
calibration_delay_done
calibrate_delay
...
```



trace-cmd examples (2/3)

- Start the function tracer and record data globally on the system

```
$ trace-cmd record -p function
```

- Use the function graph tracer but filter only `spi_*` functions

```
$ trace-cmd record -l spi_* -p function_graph
```

- Run the *irqsoff* tracer on the system:

```
$ trace-cmd record -p irqsoff
```

- Record only `irq_handler_exit/irq_handler_entry` events on the system:

```
$ trace-cmd record -e irq:irq_handler_exit -e irq:irq_handler_entry
```



trace-cmd examples (3/3)

- Visualize the data that have been acquired in `trace.dat`:

```
$ trace-cmd report
```

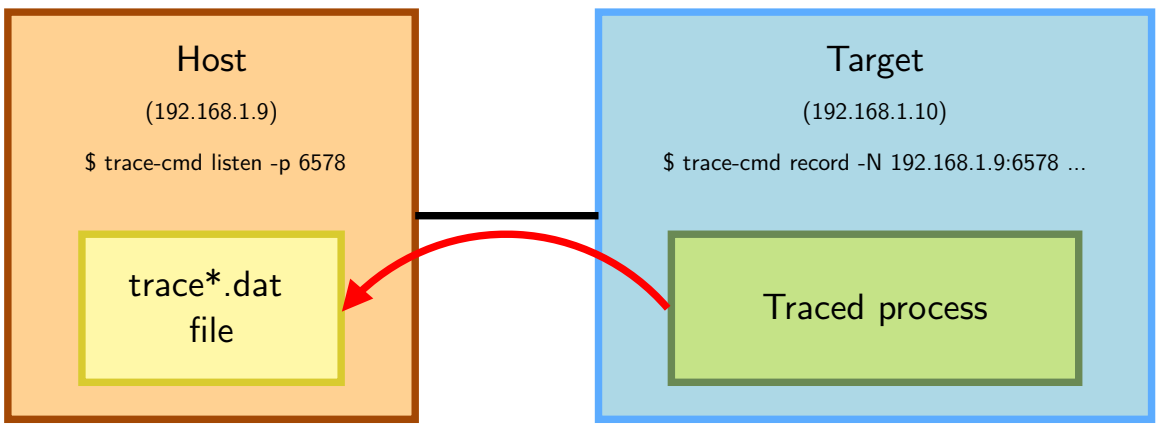
- Reset all the *ftrace* buffers and remove tracers

```
$ trace-cmd reset
```



Remote tracing with trace-cmd

- *trace-cmd* output can be quite big and thus difficult to store on an embedded platform with limited storage.
- For that purpose, a `listen` command is available and allows sending the acquisitions over the network:
  - Run `trace-cmd listen -p 6578` on the host that will be storing the traces
  - On the remote target, use `trace-cmd record -N <host_ip>:6578` to specify the host that will store the traces





trace_printk()

- trace_printk() allows to emit strings in the trace buffer
- Useful to trace some specific conditions in your code and display it in the trace buffer

```
#include <linux/ftrace.h>
void read_hw()
{
  if (condition)
    trace_printk("Condition is true!\n");
}
```

- Will display the following in the trace buffer for `function_graph` tracer

```
1)               |             read_hw() {
1)               |                /* Condition is true! */
1)   2.657 us    |             }
```



Adding ftrace tracepoints (1/2)

- For some custom needs, it might be needed to add custom tracepoints
- First, one needs to declare the tracepoint definition in a `.h` file

```
#undef TRACE_SYSTEM
#define TRACE_SYSTEM subsys

#if !defined(_TRACE_SUBSYS_H) || defined(TRACE_HEADER_MULTI_READ)
#define _TRACE_SUBSYS_H

#include <linux/tracepoint.h>

DECLARE_TRACE(subsys_eventname,
        TP_PROTO(int firstarg, struct task_struct *p),
        TP_ARGS(firstarg, p));

#endif /* _TRACE_SUBSYS_H */

/* This part must be outside protection */
#include <trace/define_trace.h>
```



Adding ftrace tracepoints (2/2)

- Then, emit tracepoint in a `.c` file using that header file

```
#include <trace/events/subsys.h>

#define CREATE_TRACE_POINTS
DEFINE_TRACE(subsys_eventname);

void any_func(void)
{
  ...
  trace_subsys_eventname(arg, task);
  ...
}
```

- See trace/tracepoints for more information



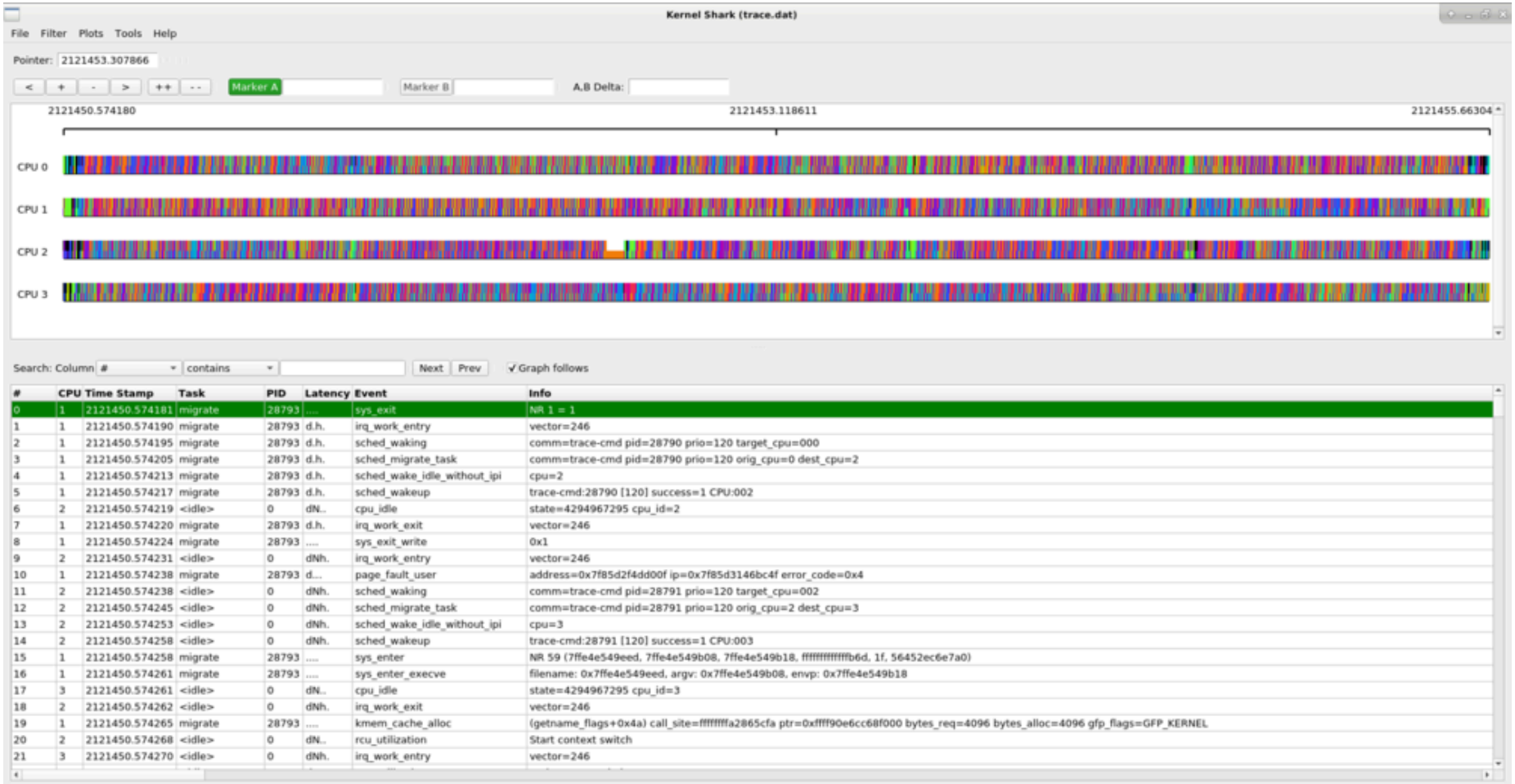
Kernelshark

- Kernelshark is a Qt-based graphical interface for processing *trace-cmd* trace.dat reports.
- Can also setup and acquire data using *trace-cmd*.
- Displays CPU and tasks as different colors along with the recorded events.
- Useful when a deep analysis is required for a specific bug.



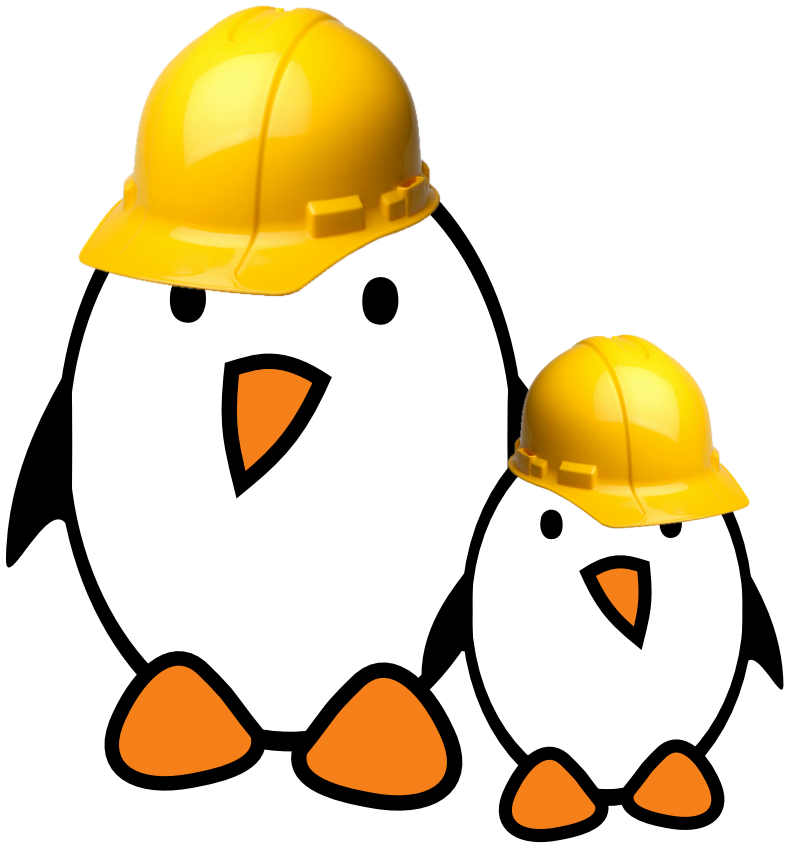


kernelshark



#	CPU	Time Stamp	Task	PID	Latency	Event	Info
0	1	2121450.574181	migrate	28793	....	sys_exit	NR 1 = 1
1	1	2121450.574190	migrate	28793	d.h.	irq_work_entry	vector=246
2	1	2121450.574195	migrate	28793	d.h.	sched_waking	comm=trace-cmd pid=28790 prio=120 target_cpu=000
3	1	2121450.574205	migrate	28793	d.h.	sched_migrate_task	comm=trace-cmd pid=28790 prio=120 orig_cpu=0 dest_cpu=2
4	1	2121450.574213	migrate	28793	d.h.	sched_wake_idle_without_ipi	cpu=2
5	1	2121450.574217	migrate	28793	d.h.	sched_wakeup	trace-cmd:28790 [120] success=1 CPU:002
6	2	2121450.574219	<idle>	0	dN..	cpu_idle	state=4294967295 cpu_id=2
7	1	2121450.574220	migrate	28793	d.h.	irq_work_exit	vector=246
8	1	2121450.574224	migrate	28793	....	sys_exit_write	0x1
9	2	2121450.574231	<idle>	0	dNh.	irq_work_entry	vector=246
10	1	2121450.574238	migrate	28793	d...	page_fault_user	address=0x7f85d2f4dd00f ip=0x7f85d3146bc4f error_code=0x4
11	2	2121450.574238	<idle>	0	dNh.	sched_waking	comm=trace-cmd pid=28791 prio=120 target_cpu=002
12	2	2121450.574245	<idle>	0	dNh.	sched_migrate_task	comm=trace-cmd pid=28791 prio=120 orig_cpu=2 dest_cpu=3
13	2	2121450.574253	<idle>	0	dNh.	sched_wake_idle_without_ipi	cpu=3
14	2	2121450.574258	<idle>	0	dNh.	sched_wakeup	trace-cmd:28791 [120] success=1 CPU:003
15	1	2121450.574258	migrate	28793	....	sys_enter	NR 59 (7ffe4e549eed, 7ffe4e549b08, 7ffe4e549b18, ffffffffffffb6d, 1f, 56452ec6e7a0)
16	1	2121450.574261	migrate	28793	....	sys_enter_execve	filename: 0x7ffe4e549eed, argv: 0x7ffe4e549b08, envp: 0x7ffe4e549b18
17	3	2121450.574261	<idle>	0	dN..	cpu_idle	state=4294967295 cpu_id=3
18	2	2121450.574262	<idle>	0	dNh.	irq_work_exit	vector=246
19	1	2121450.574265	migrate	28793	....	kmem_cache_alloc	(getname_flags+0x4a) call_site=ffffffffa2865cfa ptr=0xffff90e6cc68f000 bytes_req=4096 bytes_alloc=4096 gfp_flags=GFP_KERNEL
20	2	2121450.574268	<idle>	0	dN..	rcu_utilization	Start context switch
21	3	2121450.574270	<idle>	0	dNh.	irq_work_entry	vector=246

Practical lab - System wide profiling



Profiling a system from userspace to kernel space

- Profiling with ftrace, uprobes and kernelshark
- Profiling with perf



LTTng



LTTng

- LTTng is an open source tracing framework for Linux maintained by the EfficiOS company.
- LTTng allows understanding the interactions between the kernel and applications (C, C++, Java, Python).
  - Also expose a `/dev/lttng-logger` that can be used from any application.
- Tracepoints are associated with a payload (data).
- LTTng is focused on low-overhead tracing.
- Uses the Common Trace Format (so traces are readable with other software like babeltrace or trace-compass)





Tracepoints with LTTng

- LTTng works with a session daemon that receive all events from kernel and userspace LTTng tracing components.
- LTTng can use and trace the following instrumentation points:
  - User space LTTng tracepoints
  - Linux user space probes
  - Linux kernel system calls
  - LTTng kernel tracepoints
  - kprobes and kretprobes



Creating userspace tracepoints with LTTng

- New userspace tracepoints can be defined using LTTng.
- Tracepoints have multiple characteristics:
  - A provider namespace
  - A name identifying the tracepoint
  - Parameters of various types (int, char *, etc)
  - Fields describing how to display the tracepoint parameters (decimal, hexadecimal, etc) (see LTTng-ust manpage for types)
- Developers must perform multiple operations to use UST tracepoint: write a tracepoint provider (.h), write a tracepoint package (.c), build the package, call the tracepoint in the traced application, and finally build the application, linked with lttng-ust library and the package provider.
- LTTng provides the `lttng-gen-tp` to ease all those steps, allowing to only write a template (.tp) file.



Defining a LTTng tracepoint (1/2)

- Tracepoint template (`hello_world-tp.tp`):

```
LTTNG_UST_TRACEPOINT_EVENT(
  // Tracepoint provider name
  hello_world,

  // Tracepoint/event name
  my_first_tracepoint,

  // Tracepoint arguments (input)
  LTTNG_UST_TP_ARGS(
      char *, text
  ),

  // Tracepoint/event fields (output)
  LTTNG_UST_TP_FIELDS(
      lttng_ust_field_string(message, text)
  )
)
```

- `lttng-gen-tp` will take this template file and generate/build all needed files (.h, .c and .o files)



Defining a LTTng tracepoint (2/2)

- Build tracepoint provider:

```
$ lttng-gen-tp hello_world-tp.tp
```

- Tracepoint usage (`hello_world.c`):

```c
#include <stdio.h>
#include "hello-tp.h"

int main(int argc, char *argv[])
{
    lttng_ust_tracepoint(hello_world, my_first_tracepoint, "hi there!");
    return 0;
}
```

- Compilation:

```
$ gcc hello_world.c hello_world-tp.o -llttng-ust -o hello_world
```



Using LTTng

```
$ lttng create my-tracing-session --output=./my_traces
$ lttng list --kernel
$ lttng list --userspace
$ lttng enable-event --userspace hello_world:my_first_tracepoint
$ lttng enable-event --kernel --syscall open,close,write
$ lttng start
$ /* Run your application or do something */
$ lttng destroy
$ babeltrace2 ./my_traces
```

- You can also use trace-compass to display the traces in a GUI



Remote tracing with LTTng

- LTTng allows to record traces over the network.
- Useful for embedded systems with limited storage capabilities.
- On the remote computer, run `lttng-relayd` command

```
$ lttng-relayd --output=${PWD}/traces
```

- Then on the target, at session creation, use the `--set-url`

```
$ lttng create my-session --set-url=net://remote-system
```

- Traces will then be recorded directly on the remote computer.